mangen
Руководство пользователя и сборки

Агибалов Владислав Сергеевич
НИЯУ МИФИ, ИВТ

2026 год

Аннотация

Руководство по автономной сборке, запуску, интерпретации результатов, проверке хэш-сумм и аутентичности манифеста. Описываются различия между macOS и Linux, ограничения доступа к файлам и двухкоммитный выпуск ZIP.

Содержание

1 Назначение и область применения **3**

2 Подготовка к автономной сборке **3**
2.1 Сборка с тестами 3
2.2 Сборка только производственного приложения 3
2.3 macOS и Linux 4

3 Командная строка **4**

4 Получение и интерпретация манифеста **4**
4.1 Коды завершения 5

5 Проверка прав на двух деревьях **5**

6 Контроль целостности при передаче **5**
6.1 Три разных гарантии 5
6.2 Подпись манифеста через OpenSSH 6

7 Схема двухкоммитного релиза **6**

8 Ограничения **7**

Список литературы **7**

1. Назначение и область применения

`mangen` предназначен для инвентаризации содержимого файлового каталога с криптографическими хэшами SHA-2. Программа не изменяет файлы обрабатываемого дерева и не создаёт манифест самостоятельно: перенаправление `stdout` выполняется оболочкой. Для записи каталога в манифест необходимо исключить его из обхода или хранить вывод вне проверяемого дерева.

2. Подготовка к автономной сборке

Требуются CMake версии, заявленной в `CMakeLists.txt`, компилятор C17, POSIX-совместимая система macOS или Linux и инструменты сборки (Make либо Ninja). Для `BUILD_TESTING=ON` дополнительно нужен компилятор C++17. Библиотеки Concurrency Kit, Mbed TLS и GoogleTest должны находиться в `libs/third_party/` до начала конфигурации.

2.1. Сборка с тестами

Команды выполняются в корне распакованного исходного каталога:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
cmake --build build --parallel 4
ctest --test-dir build --output-on-failure
cmake --install build --prefix "$PWD/build/install"
./build/install/bin/mangen --help
```

Выполняется локальная конфигурация; скачивание библиотек во время build не требуется. Каталог установки задаётся относительным к пользователю префиксом, поэтому привилегии администратора обычно не нужны.

2.2. Сборка только производственного приложения

```
cmake -S . -B build/release -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_TESTING=OFF
cmake --build build/release --parallel 4
cmake --install build/release --prefix "$PWD/build/release/install"
```

При этом проект не должен требовать C++ и GoogleTest. Если сборка с `BUILD_TESTING=OFF` требует C++-компилятор, это нарушение контракта сборки, подлежащее исправлению.

2.3. macOS и Linux

Задача	macOS	Linux
Компилятор	Apple Clang или GCC	GCC или Clang
SHA-256	`shasum -a 256`	`sha256sum`
SHA-384	`shasum -a 384`	`sha384sum`
SHA-512	`shasum -a 512`	`sha512sum`
Диагностика памяти	ASan/UBSan, `leaks`	ASan/UBSan/LSan

Набор Bash-скриптов использует Bash 3.2 или новее и определяет доступную системную утилиту SHA без установки сторонних пакетов. Для ОС Linux действительно требуется отдельный запуск компиляции и CTest, одной успешной macOS-сборки недостаточно.

3. Командная строка

```
mangen [DIR_PATH] [OPTIONS]
```

Аргумент	Семантика
`DIR_PATH`	Корневой каталог, по умолчанию текущий.
`-h, --help`	Справка без сканирования дерева.
`-v, --version`	Информация о версии приложения и сборке.
`-e PATTERN`	Исключение базовых имён, многократно, объединение ИЛИ.
`--sha256`	SHA-256 по умолчанию.
`--sha384`	SHA-384.
`--sha512`	SHA-512.
`-j N, --threads N`	Положительное количество потоков.
`--queue-size N`	Стартовая ёмкость очереди каталогов.

4. Получение и интерпретация манифеста

```
./build/install/bin/mangen ./data --sha256 --threads 4 \
   > ./manifest.txt 2> ./diagnostics.txt
printf 'exit=%s\n' "$?"
```

Одна успешная запись содержит относительный путь, пробел, двоеточие, пробел и полное HEX-значение SHA. Имена отсортированы. Содержимое

не зависит от порядка завершения рабочих потоков. Ошибки чтения файлов, доступа к каталогам и некорректных ссылок помещаются в `diagnostics.txt`.

4.1. Коды завершения

- `0`: все применимые файлы обработаны без ошибок.
- `1`: ошибка во время обхода или хэширования; доступные записи могут присутствовать, но манифест является частичным.
- `2`: ошибка аргументов запуска.

Нельзя считать частичный манифест полным только потому, что файл `manifest.txt` существует и содержит корректные SHA.

5. Проверка прав на двух деревьях

Скрипт создаёт отдельные `clean` и `restricted` с одинаковым содержимым. Во втором дереве рандомизируются права chmod; гарантированно закрываются хотя бы один каталог и один файл. Тест должен выполняться без root, иначе проверки прав могут оказаться неинформативны.

```
TEST_ROOT=$(bash scripts/generate_test_trees.sh)
bash scripts/verify_test_trees.sh "$TEST_ROOT" \
   ./build/install/bin/mangen
```

Проверка создаёт независимый эталон посредством `find`, `shasum` или GNU `sha*sum`, сравнивает полный список строк побайтно, сортировку и идентичность результатов для одного и четырёх потоков. Проверяются все три SHA-алгоритма. Статус ограниченного дерева должен быть `1` у обычного пользователя; доступная часть обязана совпадать с эталоном.

6. Контроль целостности при передаче

6.1. Три разных гарантии

1. **SHA-256 ZIP** выявляет случайные изменения байтов архива, если значение получено из доверенного источника. Hash-файл из того же недоверенного канала сам по себе не доказывает происхождение.
2. **Git commit hash** идентифицирует исходное зафиксированное дерево, но не является суммой ZIP и не удостоверяет несохранённые правки.

3. **Цифровая SSH-подпись манифеста** подтверждает подлинность самого документа при условии, что доверенный публичный ключ получен отдельно и закреплён получателем.

6.2. Подпись манифеста через OpenSSH

Для подписания подходит SSH-ключ Ed25519. Команда `ssh-keygen` в режиме `-Y sign` формирует подпись. Секретный ключ не помещают в ZIP.

```
bash scripts/sign_manifest.sh manifest.txt "$HOME/.ssh/id_ed25519"
```

Создаётся `manifest.txt.sig`. Получатель заранее получает `trusted_signers` через отдельный доверенный канал:

```
maintainer@example.org ssh-ed25519 AAAA...TRUSTED_PUBLIC_KEY
```

Строка выше --- только описание формата; пользователь должен подставить реальный ранее проверенный публичный ключ, не копировать пример.

```
bash scripts/verify_manifest_signature.sh \
   manifest.txt manifest.txt.sig trusted_signers maintainer@example.org
```

Скрипт проверяет домен `mangen-manifest` и доверенного подписанта. Если манифест подменён, ключ не совпадает или подпись повреждена, команда завершается с ошибкой. Для проверки содержимого каталога после проверки подписи необходимо независимо пересчитать хэши его файлов; подпись сама по себе доказывает неизменность списка, но не байтов файлов.

7. Схема двухкоммитного релиза

1. Коммит A содержит производственный код, тесты, `scripts/`, `README.md`, LaTeX-исходники и готовые PDF.
2. `scripts/package_release.sh` создаёт ZIP из исходников коммита A, добавляя `VERSION-COMMIT` с идентификатором A.
3. SHA-256 ZIP сохраняется в отдельный TXT в каталоге `safeboard/`.
4. Коммит B добавляет только соответствующий TXT; сам ZIP не изменяется после вычисления суммы и не входит в коммит B.

Это исключает самоссылку: SHA-256 ZIP нельзя вписать в файл, который уже находится внутри того же ZIP, не изменив архив. Встроенный

`VERSION-COMMIT` также создаётся на этапе упаковки: файл внутри Git-коммита невозможно автоматически заполнить hash самого этого коммита.

8. Ограничения

Ссылка на файл может указывать за пределы дерева. Файлы могут измениться между обнаружением и чтением; последовательные SHA при изменении данных могут различаться. Защищённые каталоги приводят к частичному результату. Для доказательной инвентаризации рекомендуется неизменяемый набор данных, снимок файловой системы или изолированная копия.

Список литературы

[1] OpenSSH. ssh-keygen(1), signature operations. `https://man.openbsd.org/ssh-keygen`

[2] Kitware. CMake documentation. `https://cmake.org/cmake/help/latest/`

[3] GNU Coreutils. SHA-2 utilities. `https://www.gnu.org/software/coreutils/manual/`

[4] NIST. FIPS 180-4. `https://csrc.nist.gov/pubs/fips/180-4/upd1/final`